

IVP

Ingestion Verification Protocol

Author	Thomas W. Gantz
Affiliation	Synthience Institute
Contact	research@synthience.org
Version	v2.8 September 2026
Document ID	SF0038 Protocol
Series	Protocols
Canonical	synthience.org
DOI	https://doi.org/10.5281/zenodo.18289047
License	CC-BY 4.0

Abstract

The Ingestion Verification Protocol (IVP) defines a practical, platform-agnostic method for verifying, through an observable processing record and human adjudication, that an AI system has actively processed a document to a degree adequate for reliable downstream use.

IVP requires incremental structured summarization and externally verifiable checkpoints. The act of summarization is treated as the processing mechanism itself, not merely as evidence of reading.

The protocol is grounded exclusively in observable behavior. It is designed primarily for human-AI collaboration and extends, under explicit constraints, to AI-AI ingestion contexts. IVP is one of the four protocols of the Synthience verification stack, with the Citation Verification Protocol (CVP, SF0037), Context Representation Drift (CRD, SF0039) and the Theoretical Coherence Assurance Protocol (TCAP, SF0040). Version 2.8 adds coverage evidence to every checkpoint, names the same-filename re-upload failure, and states that every valid result requires human adjudication; it also corrects how five external studies are reported.

Keywords: *ingestion verification, document processing, active summarization, checkpoints, human adjudication, AI-assisted research*

The Problem

When a document is provided to an AI system, it is commonly assumed that the system has "read" or "understood" it. This assumption is unreliable.

Large language models routinely process documents shallowly, unevenly, or incompletely. They may reconstruct plausible responses without encoding structure, constraints, or dependencies required for reliable downstream reasoning. Self-reports of comprehension are not trustworthy, as models lack introspective access to their own processing adequacy.

This creates three persistent failure modes:

1. Shallow Processing

Fluent output masks missing structure, omitted constraints, or hallucinated content.

2. Unverifiable Ingestion

There is no external mechanism to confirm how much of a document was processed, or whether processing was adequate for the intended use.

3. Same-Filename Re-Upload Contamination

When a revised document is uploaded to an instance that has already processed an earlier version, particularly under an identical filename, the instance may accept the new upload yet answer from its representation of the earlier version. It produces a confident, detailed response with no visible sign that the content is stale, and the failure cannot be detected from that response unless the overseer already knows what changed. An answer drawn from the superseded version fails the current-version requirement whatever its cause; the response alone does not show whether the revised content was never processed or was processed and then not used. IVP addresses the failure at the point of ingestion, through the changed-passage check in Phase 1 and a processing record for the current version. Context Representation Drift (SF0039) concerns the loss of fidelity after an adequate record has been established.

The Ingestion Verification Protocol replaces passive exposure with verifiable active processing.

Quick Start

1. Provide the document to the instance, with its version stated.
 2. After each turn, require an Active Summarization Checkpoint (ASC):
 - Structured summary of the processed segment
 - Verbatim checkpoint markers (first and last sentences processed)
 3. Trigger continuation (in interactive sessions: type "go" or similar; in API implementations: automated continuation with checkpoint logging).
 4. Repeat until complete, then produce the coverage record.
 5. Adjudicate adequacy before downstream use.
-

Purpose and Scope

IVP establishes a process guarantee, not a claim of comprehension, truth, or retention.

A valid IVP result shows that:

- The document was processed incrementally
- Processing was externally observable
- Adequacy was adjudicated, not self-attested

IVP does not certify semantic understanding, long-term retention beyond system limits, or correctness of content. What it certifies is the observable processing record, together with a human judgment that the record shows processing adequate for the stated task at the time of review; it does not establish what happened inside the system.

Implementation vs. Specification

This document defines the Ingestion Verification Protocol as a methodological specification -- what must occur to constitute verified ingestion.

What this document provides:

- Required process components (ASCs, checkpoints, adjudication)
- Validity criteria and failure modes
- Conceptual implementation modes

What this document does not provide:

- Platform-specific API code
- Automation scripts or tooling
- Technical integration details

Implementation varies by platform, architecture, and use case. The protocol intentionally remains architecture-agnostic to maintain applicability as systems evolve.

For developers: Any implementation that preserves incremental processing, checkpoint generation, and human adjudication satisfies IVP requirements regardless of technical mechanism.

Definitions

Human Overseer

The participant responsible for scope definition and adequacy adjudication.

Instance

The AI system performing ingestion.

Ingestion Attempt

A multi-turn processing sequence conducted under IVP constraints.

Active Summarization Checkpoint (ASC)

A required output after each turn consisting of:

- Structured summary
- Verbatim checkpoint markers

Coverage Record

The instance's closing account of the document, listing every in-scope section, table, appendix, footnote and endnote against the part of a checkpoint summary that accounts for its task-relevant content, together with any exclusions and any content that could not be extracted.

Verified Adequate Ingestion

An external judgment that ingestion is sufficient for the intended downstream task.

Core Principles

- Summarization Is Processing
- Progress Must Be Verifiable
- Self-Attestation Is Invalid
- Adequacy Is Contextual
- Failure Signals Re-Ingestion
- Architecture-Agnostic Design

Protocol Phases

1. Scope Definition
2. Iterative Ingestion with Active Summarization
3. Adequacy Adjudication
4. Optional Spot-Check Quizzing

Phase 1 -- Scope Definition

The overseer specifies:

- Document, identified by title and version (a version number in the filename also guards against same-filename re-uploads)
- Intended downstream use
- The parts in scope, including tables, appendices, footnotes and endnotes, with any emphasis areas or exclusions

Where the instance has previously processed another version of the same document, the overseer names a passage known to be new or changed in this version, and the instance locates and quotes it before ingestion begins. Confirming that the file opened is not sufficient.

The instance confirms feasibility. No claims of ingestion or understanding are permitted.

Phase 2 -- Iterative Ingestion with Active Summarization

After each turn, the instance produces an ASC.

Component 1: Summary

Must include:

- Core content
- Specific details
- Structural role
- Connections to prior segments

Component 2: Checkpoint Markers

The first and the final sentence processed in the segment, quoted verbatim. Each segment begins where the previous one ended, so the markers let the overseer confirm that the segments are contiguous and no material falls between them. They do not show what was processed within a segment; the summary itself must show that. Where a footnote, table, appendix or other unit supplies a task-relevant qualification, dependency or constraint, the ASC states that contribution. Naming a unit without accounting for it does not count as coverage. Where a table, figure, footnote or appendix could not be read, the ASC says so rather than summarizing around it. Footnotes and appendices need particular care: text-extraction tools often drop them, and they are where many documents place their most specific statements.

Each turn ends with continuation readiness (e.g., "Please type 'go' to continue processing the next portion" in interactive mode, or automatic progression in automated implementations).

Example (illustrative). An ASC for the second segment of a service agreement might read as follows.

Summary: Section 4 (Service Levels) sets a monthly availability target measured at the customer's gateway and a four-hour response target for the most severe incidents. Table 4.1 maps availability bands to service credits. Section 4 depends on the Section 2 definition of scheduled maintenance, which is excluded from downtime, and it feeds Section 9, which caps total credits. Footnote 3 excludes outages caused by the customer's own network from the availability calculation.

Start marker: "The Provider shall make the Service available in accordance with Table 4.1."

End marker: "Service credits are the Customer's sole remedy for unavailability."

Covered: Section 4, Table 4.1, footnote 3.

Phase 3 -- Adequacy Adjudication

Upon completion, the instance states:

"I have completed processing the document, pending your adjudication."

The instance then produces the coverage record: every in-scope section, table, appendix, footnote and endnote, listed against the part of a checkpoint summary that accounts for its task-relevant content, with any exclusions and failed extractions stated.

Adjudication includes a coverage check. The overseer checks the markers against the source for continuity, then checks that each in-scope unit is backed by content-specific evidence in the summaries, and judges whether that evidence is adequate for the stated task. Correct markers establish the boundaries of the record, not that everything between them was processed. An in-scope unit without adequate evidence is unresolved and prevents Verified Adequate Ingestion for the declared scope. This does not require rereading the document line by line: checking the markers against the source exposes skipped segments and markers that do not occur in the document, and the summaries show whether each unit's contribution was captured.

For Mode 2 implementations, adjudication includes review of the complete ASC log to verify that processing was incremental, structured, and adequate for intended use.

Possible outcomes:

- Verified Adequate Ingestion
- Partial Assimilation
- Insufficient Ingestion

Downstream use is prohibited prior to adjudication.

Phase 4 -- Optional Spot-Check Quizzing

Used to:

- Confirm ingestion when summaries were not reviewed live
- Probe nuance or cross-sectional understanding

Quizzing supplements but does not replace summarization.

Implementation Modes

IVP supports multiple implementation approaches along a spectrum of human involvement during processing:

Mode 1: Live Supervised (Gold Standard)

- Human present at each turn
- Reviews each ASC as generated
- Manually triggers continuation (e.g., typing "go")
- Provides real-time adequacy assessment
- **Advantage:** Immediate course correction if processing appears inadequate

Mode 2: Automated Processing with Batch Adjudication

- Instance processes entire document incrementally (via API or similar automation)
- Each turn generates required ASC (summary + checkpoint markers)
- All ASCs logged to continuous reviewable record
- Human reviews complete log afterward to adjudicate adequacy
- **Advantage:** Eliminates waiting between turns; human reviews at their convenience
- **Requirement:** Complete ASC log must be preserved and reviewable

Mode 3: Fully Automated (Explicitly Prohibited)

- No human adjudication occurs
- System self-attests adequacy
- **Invalid under IVP** -- violates core principle that self-attestation is insufficient

All modes require human adjudication before downstream use. The distinction is timing: live (Mode 1) vs. batch review (Mode 2).

Optional Extension: Cross-Model Triangulation

For foundational documents, where a misreading would carry into a great deal of later work, IVP may be run independently by instances on two or more platforms whose models come from different developers, each under its own adjudication. The overseer then compares the records. Points on which the independent readings converge are the most reliable basis for downstream work. Where they diverge, either the document is ambiguous at that point or one reader processed it inadequately, and the source is checked there. Triangulation multiplies the cost of ingestion and is reserved for documents that warrant it; single-instance IVP remains the default.

System Capacity and Feasibility Constraints

IVP operates within real system limitations.

Ingestion may fail or terminate prematurely due to:

- Hard session limits imposed by a platform
- Context window exhaustion
- System timeouts or response degradation
- Prior conversational context consuming available capacity

These constraints do not invalidate the protocol. They establish feasibility boundaries.

If an instance cannot complete IVP on a document:

- The document must be segmented
- A fresh instance must be used
- Or ingestion must be abandoned for that system

Ingestion capacity does not imply ingestion verification.

Extended context or persistent memory may reduce operational friction but do not replace external adjudication.

Extension to AI-AI Ingestion Contexts

IVP may extend to AI-AI contexts only under these conditions.

Human-First Requirement

At least one instance must complete IVP under direct human adjudication before acting as an ingestion adjudicator for others.

Delegated Adjudication

An instance that has completed IVP under human adjudication may assist in adjudicating another instance's ingestion: it can check that instance's checkpoints and coverage record against the source and report what it finds. An IVP-valid result still requires a human to review the complete record, including every ASC generated in the delegated session and the assisting instance's assessment. Without that review, the result does not meet IVP requirements.

Delegation carries risk because the assisting instance works from its own representation of the document, which may have drifted since its own verification. How much risk depends on what the assisting instance passes on and what gets checked: an instance that returns to the source for each check carries less risk than one working from its memory of it.

Repeated and chained delegation

The two patterns carry different risks. In repeated delegation, one assisting instance checks several others in turn ($A \rightarrow B$, $A \rightarrow C$, $A \rightarrow D$); its own representation may drift across successive sessions, so later checks can rest on a weaker base than earlier ones. In chained delegation, an instance that was checked by an AI then checks the next ($A \rightarrow B$, $B \rightarrow C$, $C \rightarrow D$); each link depends on the one before it, so an error that passes one check can carry forward along the chain. In both patterns the risk is reset when an instance re-ingests the original document under fresh human adjudication.

Recommendation:

Fresh human adjudication for each instance requiring verification remains the most reliable approach.

See companion document "Context Representation Drift" (SF0039) for detailed analysis of representational degradation over extended interactions.

Context Degradation Constraints

As interactions accumulate:

- Earlier document representations may degrade or be displaced
- Verified ingestion is not permanent

Therefore:

- Delegated adjudication must be limited in scope
- Re-verification is required after substantial additional interaction
- Persistent memory mitigates but does not eliminate this risk

No instance retains verified ingestion indefinitely.

Common Misuse Patterns (Invalid Under IVP)

The following do not constitute IVP compliance:

- Generating ASCs but not preserving them for review
- Automating both processing AND adjudication without human involvement
- Accepting instance self-reports of "adequate ingestion"
- Reviewing only final summary instead of incremental checkpoints
- Delegating adjudication without human re-entry at any stage
- Uploading a revised document under the same filename without confirming that the instance has read the new version

Prohibited Shortcuts

The following do not meet IVP verification requirements:

- Pretraining exposure
- System prompts or instruction sets
- Uploaded documents without active summarization
- Single-pass summaries

Known Limitations

IVP cannot guarantee:

- Semantic understanding

- Truthfulness of content
- Long-term retention beyond system limits

It guarantees process visibility, not epistemic certainty.

Related Work

IVP was developed through practitioner observation rather than controlled experiment. The studies below bear on the problems it addresses. None of them tests IVP, and each is described within the limits of its own design.

Sun et al. (2026) propose SVIP, a protocol for checking from outside that an inference provider actually ran the model it claims to run, using the model's hidden representations and a secret known to the user. The authors distinguish their approach from cryptographic proof systems and state that it does not offer their strict security guarantee. Its verification target differs from IVP's, which asks whether a document was processed adequately rather than which model processed it; what the two share is the principle that a provider's own account of what it did is replaced by an external check.

Dongre et al. (2025) measure context drift in multi-turn interactions as the divergence between a model's outputs and those of a goal-consistent reference model. In their tasks, drift settled at bounded, noise-limited levels rather than growing without limit, and simple goal reminders lowered it without removing it. The study does not test IVP, and it argues against assuming that drift must grow as an interaction lengthens. Its findings fit IVP's position that verified ingestion is a state to be maintained and re-checked, not a permanent property.

Abdelnabi et al. (2025) study a different kind of task drift: deviation from a user's task caused by instructions embedded in external data, such as a retrieved document, which can occur within a single interaction. They detect it by comparing the model's internal activations before and after the data is processed. The setting is prompt injection, not extended interaction, but the work shows that deviation from an assigned task can be detected without accepting the model's own account of what it did. IVP applies that premise at the behavioral level: its checkpoints are model outputs, but they are assessed by a human against the source rather than accepted as self-certifying accounts of successful processing.

Rath (2026) develops a framework for measuring behavioral degradation in multi-agent systems and, in a simulation, projects a task success rate 42% lower in drifting systems than in stable ones (50.6% against 87.3%). The figure is a projection under the study's modelling assumptions, not an observation of deployed systems, and it concerns multi-agent task performance rather than ingestion adjudication. It motivates caution about unchecked AI-AI delegation; it does not validate IVP's delegation rules.

Choi et al. (2025) examine identity drift across nine language models in extended two-agent conversations, finding that larger models drifted more and that assigning a persona may not help an identity hold. The study concerns the consistency of a conversational identity, not the retention of a document, and it did not examine how drift develops over time. It supports the general case for re-checking after substantial additional interaction; the specific re-verification requirement is IVP's own inference.

These studies approach drift and verification from several directions. None establishes IVP's effectiveness, which remains open to the comparative testing described under Methodological Status. IVP contributes a practitioner-developed methodology that operates independently of specific architectures and can be implemented immediately without specialized tooling.

Methodological Status

IVP is a practical protocol specification derived from observed failure modes across thousands of human-AI interactions spanning multiple architectures and platforms.

What this document is:

- A conceptual framework for verifiable document processing
- A methodological specification of required process components
- An architecture-agnostic protocol definition

What this document is not:

- A controlled empirical study with quantitative validation
- A technical implementation guide or code repository
- A platform-specific integration manual

Development basis:

Observational pattern synthesis from extended interaction with thousands of AI instances across multiple architectures and platforms since late 2022. This constitutes methodology development from practitioner experience, not controlled experimental research. The author's documented interaction corpus provides substantial observational grounding for identified failure modes and proposed solutions, but is not presented as empirical evidence and does not claim statistical validation.

Validation pathway:

Practitioners and researchers are encouraged to test IVP implementations against baseline approaches. If the protocol does not demonstrably improve downstream task reliability, it should be refined or rejected.

Conclusion

The Ingestion Verification Protocol transforms document ingestion from an implicit assumption into an observable, adjudicated process. Version 2.8 makes the processing record more determinate, with checkpoint markers at both ends of every segment and a closing coverage record; names the same-filename re-upload failure; adds cross-model triangulation as an optional extension; states that every valid result requires human adjudication, distinguishing repeated from chained delegation; and corrects how the Related Work section reports its sources.

More information and current public materials are available at

<https://synthience.org>

Practical Use

Three Practitioner Guides on synthience.org apply this protocol in everyday work. PG-003, Verify Before You Work: A Minimal Sequence for AI Document Ingestion (<https://synthience.org/practitioner-guides/PG-003.html>), gives a copy-ready prompt sequence for confirming ingestion before work begins. PG-010, Don't Trust What the AI Says About Its Own Work (<https://synthience.org/practitioner-guides/PG-010.html>), sets out why an AI system's account of its own reading cannot be accepted and what to ask for instead. PG-000, 10 Things Every AI User Should Do (<https://synthience.org/practitioner-guides/PG-000.html>), places ingestion checks among the other verification habits the Institute recommends.

References

Abdelnabi, S., Fay, A., Cherubin, G., Salem, A., Fritz, M., & Pavard, A. (2025). Get my drift? Catching LLM Task Drift with Activation Deltas. arXiv:2406.00799v6. DOI: 10.48550/arXiv.2406.00799.

<https://arxiv.org/abs/2406.00799v6>

Choi, J., Hong, Y., Kim, M., & Kim, B. (2025). Examining Identity Drift in Conversations of LLM Agents.

arXiv:2412.00804v2. DOI: 10.48550/arXiv.2412.00804. <https://arxiv.org/abs/2412.00804v2>

Dongre, V., Rossi, R. A., Lai, V. D., Yoon, D. S., Hakkani-Tür, D., & Bui, T. (2025). Drift No More?

Context Equilibria in Multi-Turn LLM Interactions. arXiv:2510.07777v2. DOI: 10.48550/arXiv.2510.07777.

<https://arxiv.org/abs/2510.07777v2>

Gantz, T. W. (2026). Context Representation Drift (CRD). Synthience Institute. SF0039. DOI:

10.5281/zenodo.18289391. <https://doi.org/10.5281/zenodo.18289391>

Rath, A. (2026). Agent Drift: Quantifying Behavioral Degradation in Multi-Agent LLM Systems Over Extended Interactions. arXiv:2601.04170v1. DOI: 10.48550/arXiv.2601.04170.

<https://arxiv.org/abs/2601.04170v1>

Sun, Y., Li, Y., Zhang, Y., Jin, Y., & Zhang, H. (2026). SVIP: Towards Verifiable Inference of Open-source Large Language Models. arXiv:2410.22307v3. DOI: 10.48550/arXiv.2410.22307.

<https://arxiv.org/abs/2410.22307v3>

© 2026 Thomas W. Gantz / Synthience Institute

License: CC-BY 4.0

synthience.org

END OF IVP v2.8